AI Grievance Assistant – POC Architecture

Oct 14, 2026 · @Simax

Overview

The POC proves one thing: an AI assistant can take a spoken grievance, route it to the right department and officer, and let leadership query the backlog in plain language. It is a "Jarvis" embedded in one app, not across the whole desktop.

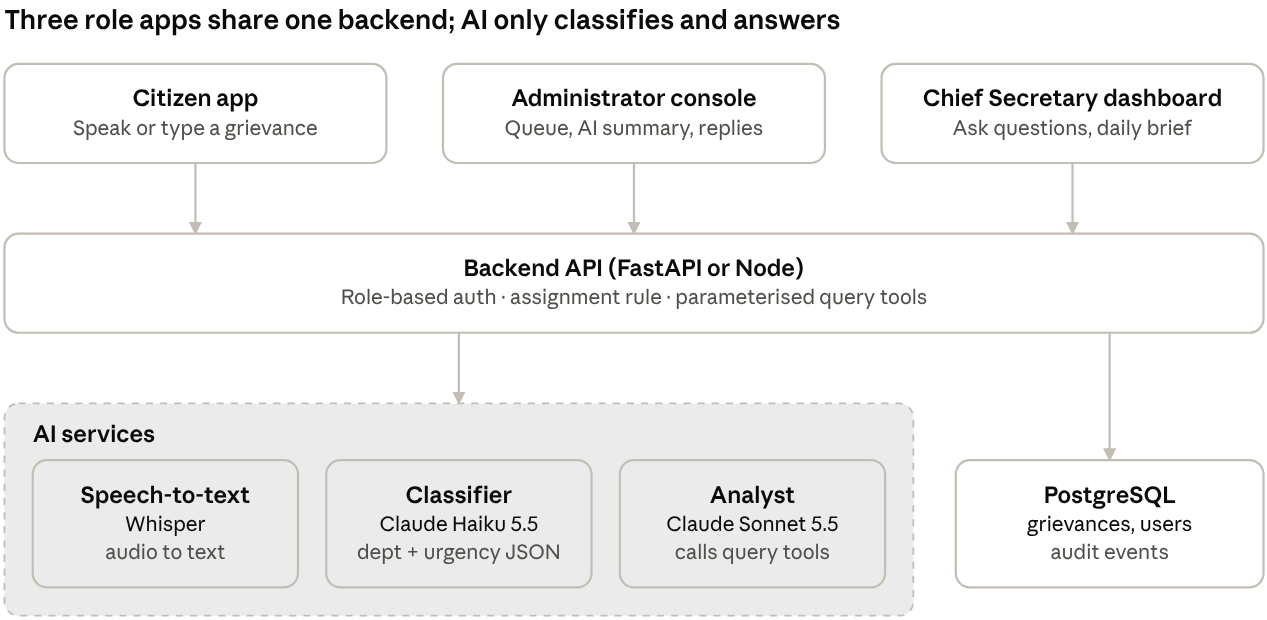
Role	What they do	What the AI does for them
Citizen (common man)	Speaks or types a grievance with their location	Transcribes, extracts details, picks the department, assigns an administrator
Administrator	Reviews and resolves assigned grievances	Summarises each case, suggests next steps, drafts replies to the citizen
Chief Secretary	Monitors the whole system	Answers questions like "top grievances today" or "worst district this week"

In scope for the POC: voice and text intake, auto-classification, auto-assignment, an admin queue, and a natural-language Q&A box for the Chief Secretary.

Out of scope for now: real government SSO, SMS/WhatsApp intake, multilingual dialects beyond 2 languages, and escalation SLAs.

Architecture

All three roles use one backend; the AI sits behind it and never touches the database directly.



POC architecture · 3 role apps, 1 backend, 4 services

The backend owns every decision that must be reliable (who gets assigned, what SQL runs); the models only interpret language and write text.

Flow 1: Citizen files a grievance by voice

The citizen taps a mic button, speaks, confirms a summary, and gets a ticket number. Routing happens without any human in the loop.

1. **Record.** The app captures audio in the browser or phone and sends it to the backend.
2. **Transcribe.** A speech-to-text model converts audio to text, with language detection.
3. **Extract.** The LLM pulls structured fields out of the text: issue summary, location (district, block, village or ward), category, urgency, and any names or dates mentioned.
4. **Confirm.** The app reads or shows the summary back: "You're reporting a broken water pipe in Ward 12, Madurai district. Is that right?" The citizen confirms or corrects.
5. **Classify.** The LLM picks one department from a fixed list you supply, plus a confidence score and a one-line reason.
6. **Assign.** Plain code (not AI) picks the administrator: the officer for that department and district with the fewest open cases.
7. **Notify.** The citizen gets a ticket ID; the administrator sees the case in their queue.

Key design choice: the LLM only chooses from your department list, returned as strict JSON. It never invents a department. If confidence is below a threshold (say 0.7), the case goes to a "needs triage" queue for a human.

Example of what the classifier returns:

```
{
  "summary": "Drinking water pipe burst, no supply for 3 days",
  "department": "Water Supply & Drainage",
  "district": "Madurai",
  "locality": "Ward 12",
  "urgency": "high",
  "confidence": 0.92,
  "reason": "Mentions pipe burst and no drinking water"
}
```

Flow 2: Administrator works the queue

The administrator gets a sorted queue and an AI co-pilot on each case; the AI drafts, the human decides.

- **Queue view:** open cases for their department and district, sorted by urgency, then age.
- **Case summary:** a 2–3 line AI summary plus the original transcript and audio, so nothing is lost.
- **Suggested next step:** e.g. "Assign field inspection" or "Request photo from citizen".
- **Draft reply:** the AI writes a status update to the citizen in their language; the admin edits and sends.
- **Reassign:** if the AI routed wrongly, the admin moves it to another department. Log every correction; it becomes your evaluation data.
- **Status:** New → In progress → Resolved → Closed (or Rejected with reason).

Flow 3: Chief Secretary asks questions in plain language

The Chief Secretary types or speaks a question; the AI calls a small set of safe, pre-built query tools and answers with numbers and a chart.

Use **tool calling, not free-form SQL**, for the POC. Letting an LLM write raw SQL against a government database is risky and hard to audit. Instead, expose a handful of functions the model can call:

Tool	Inputs	Answers questions like
`count_grievances`	date range, district?, department?, status?	"How many grievances came in today?"
`top_categories`	date range, district?, limit	"What are the top grievances this week?"
`by_district`	date range, department?	"Which district has the most water complaints?"
`pending_overdue`	days threshold, department?	"What's been pending more than 15 days?"
`admin_performance`	date range	"Which officers are resolving fastest?"
`get_samples`	filters, limit	"Show me a few examples from Salem."

How a question is answered:

1. The question plus the tool list go to the LLM.
2. The LLM calls one or more tools with filled-in parameters, e.g. `by_district(date_range="this_week", department="Water Supply")`.
3. Your backend runs a fixed, parameterised SQL query and returns rows.
4. The LLM writes a short answer ("Madurai leads with 142, up 30% on last week") and the UI renders a chart from the rows.

The Chief Secretary can also get a **daily brief**: a scheduled job runs the same tools each morning and the LLM writes a 5-line summary.

If you later need open-ended questions, add a read-only text-to-SQL tool against a reporting view, with row limits and query logging.

Models and tech stack

Use a fast, cheap model for high-volume intake and a stronger model for the Chief Secretary's analysis.

Job	Recommended	Why	Alternatives
Classify + extract (Flow 1)	Claude Haiku 5.5 (`claude-haiku-5-5`)	High volume, structured JSON, low cost and latency	Claude Sonnet 5.5 if accuracy on messy speech is low
Admin summary + draft reply (Flow 2)	Claude Haiku 5.5	Short summaries and drafts	Sonnet 5.5 for sensitive replies
Chief Secretary Q&A + daily brief (Flow 3)	Claude Sonnet 5.5 (`claude-sonnet-5-5`)	Multi-step tool calling and good written analysis	Opus 5.5 only if questions get very complex
Speech-to-text	OpenAI Whisper (self-hosted, open source)	Multilingual, free to run, works offline	Google Cloud Speech-to-Text, Azure Speech; for Indian languages, Bhashini / AI4Bharat models
Text-to-speech (read-back)	Browser Web Speech API for the POC	Zero setup	Google Cloud TTS, Azure TTS for better regional voices

Model names and pricing change often; confirm current models at docs.claude.com before you build. The STT/TTS options are from memory, so check their current Indian-language support.

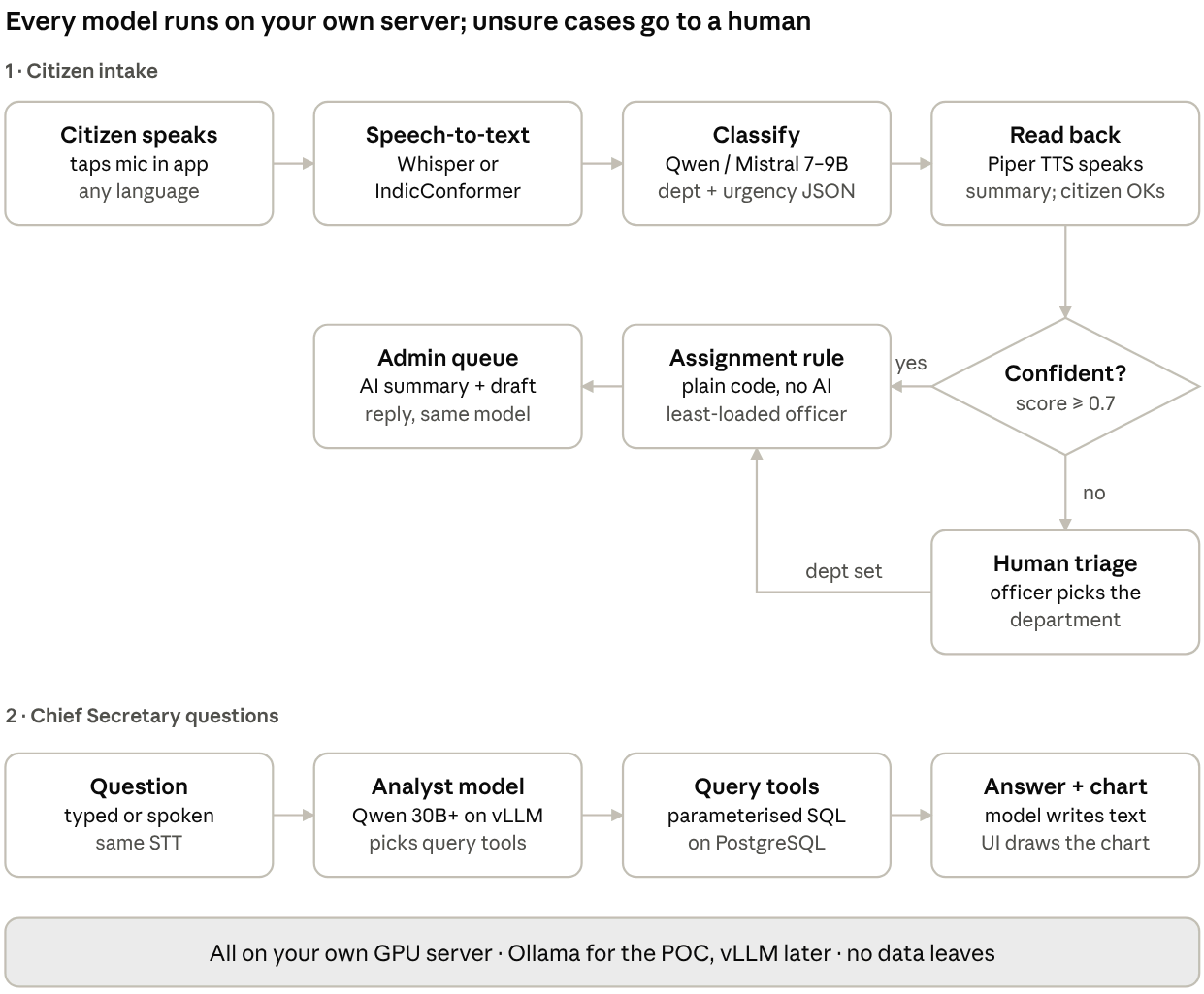
Suggested POC stack:

Layer	Choice
Frontend	React (web) or React Native / Flutter (mobile)
Backend	Python FastAPI or Node.js (Express)
Database	PostgreSQL (PostGIS later for maps)
AI calls	Anthropic SDK with tool use and JSON output
Auth	Simple role-based login: citizen, admin, chief_secretary

Layer	Choice
Charts	Recharts or Chart.js on the dashboard
Hosting	Single VM or Docker Compose for the demo

Fully open-source option

The whole POC can run on open models hosted on your own server, so citizen data never leaves your infrastructure. The architecture stays the same; only the AI boxes change.



open-source flow · citizen intake and Chief Secretary Q&A

The small model handles high-volume intake; only the Chief Secretary's multi-step questions need the larger model.

Job	Open-source option	Notes
Speech-to-text	Whisper (MIT) or AI4Bharat IndicConformer	AI4Bharat is usually stronger on Indian languages and dialects
Classify + extract	7–9B model: Qwen, Mistral or Llama	Give it the department list and a few examples; force JSON output
Admin summary + draft reply	Same 7–9B model	No second model needed
Chief Secretary Q&A	30B+ model, e.g. Qwen family	Multi-step tool calling is where small models fail
Text-to-speech	Piper, or AI4Bharat Indic TTS	Piper is light; AI4Bharat covers more Indian voices
Serving	Ollama (POC), vLLM (production)	Both expose an OpenAI-style API, so the backend code barely changes

Hardware: one GPU with 16–24 GB runs Whisper plus a quantised 8B classifier. The 30B+ analyst needs roughly 24–48 GB more, depending on quantisation. A CPU works for testing, slowly.

Licences: "open source" here mostly means open weights. Qwen and many Mistral models are Apache 2.0; Llama and Gemma have their own conditions. Check each licence before a government deployment.

Trade-off: open models classify well but are less reliable at multi-step tool calling than hosted frontier models. Run the same 50–100 labelled grievances through both and compare. A hybrid also works: open models for intake (where personal data is), a hosted model only for aggregate Chief Secretary queries.

Model families listed reflect knowledge up to mid-2026; check the latest releases before choosing versions.

Data model

Five tables are enough for the POC.

Table	Key columns
`users`	id, name, phone, role (citizen / admin / chief_secretary), district_id, department_id
`departments`	id, name, description (fed to the classifier as the allowed list)
`districts`	id, name, state
`grievances`	id, citizen_id, transcript, audio_url, summary, department_id, district_id, locality, urgency, ai_confidence, ai_reason, assigned_admin_id, status, created_at, resolved_at
`grievance_events`	id, grievance_id, actor_id, action (created / reassigned / status_changed / replied), note, created_at

Keep the original transcript and the AI's output side by side. `grievance_events` gives you an audit trail and shows where the AI routed wrongly.

Build plan, risks and success criteria

Build the riskiest piece first: classification accuracy on real, messy speech. If routing is wrong, nothing else matters.

Build order (about 3–4 weeks for one developer):

1. Collect 50–100 sample grievances (real or realistic) with the correct department labelled.
2. Build text-only classification and measure accuracy against those labels.
3. Add voice: STT in front of the classifier, then re-test on spoken versions.
4. Build the auto-assignment rule and the admin queue.
5. Build the Chief Secretary tools and Q&A box, seeded with ~1,000 synthetic grievances so the answers look meaningful.
6. Add the daily brief and a simple dashboard.

Risks and mitigations:

Risk	Mitigation
Wrong department chosen	Fixed department list, confidence threshold, human triage queue, log reassignments
Poor transcription of dialects or noisy audio	Confirmation step before submit; allow typed correction
AI makes up numbers for the Chief Secretary	Answers come only from tool results; show the source rows/chart beside the text
Personal data exposure	Mask phone numbers in AI prompts; role-based access; check data-residency rules for government data
Unsafe SQL	Parameterised tools only; read-only DB user

The POC succeeds if:

- [ ] Department routing is correct for at least 85% of test grievances
- [ ] A spoken grievance becomes an assigned ticket in under 10 seconds
- [ ] The Chief Secretary gets correct answers to 10 scripted questions
- [ ] An administrator can reassign, reply and close a case end to end